# session, cookie

## 말고 JWT

권기웅

# 1. JSON Web Token

## 1.1 일단 준비

### 1.1.1 back-end

SpringMVC 프로젝트

pom.xml 에

```
<dependency>
    <groupId>org.codehaus.jackson</groupId>
    <artifactId>jackson-mapper-asl</artifactId>
    <version>1.9.13</version>
</dependency>

<dependency>
    <groupId>io.jsonwebtoken</groupId>
    <artifactId>jjwt</artifactId>
    <version>0.9.1</version>
</dependency>
```

## 1.1.2 front-end

jwtGogogo.js

```
import React, { useState } from "react";

const JWTGogogo = () => {
  const [name, setName] = useState("");
  const [price, setPrice] = useState("");

  return (
    <>
      이름
      <input
        value={name}
        onChange={(e) => {
          setName(e.target.value);
        }}
      />
      <br />
      가격
      <input
        value={price}
        onChange={(e) => {
          setPrice(e.target.value);
        }}
      />
    </>
  );
};

export default JWTGogogo;
```

App.js

```
import "./App.css";
import JWTGogogo from "./jwtGogogo";

function App() {
  return <JWTGogogo />;
}

export default App;
```

# 1.2 JWT 만들기

## 1.2.1 back-end

Token.java

```
public class Token {
    private String token;

    public Token() {
        // TODO Auto-generated constructor stub
    }

    public Token(String token) {
        super();
        this.token = token;
    }

    public String getToken() {
        return token;
    }

    public void setToken(String token) {
        this.token = token;
    }
}
```

Snack.java

```
public class Snack {
    private String name;
    private int price;

    public Snack() {
        // TODO Auto-generated constructor stub
    }

    public Snack(String name, int price) {
        super();
        this.name = name;
        this.price = price;
    }

    public String getName() {
        return name;
    }

    public void setName(String name) {
        this.name = name;
```

JWTDAO.java

```java
@Service
public class JWTDAO {

    public Token make(Snack s) {
        Date now = new Date();
        String jwt = Jwts.builder()
                .signWith(SignatureAlgorithm.HS256, "dhffpxlqlahtQhk")
                .setExpiration(new Date(now.getTime() + Duration.ofSeconds(10).toMillis()))
                .claim("name", s.getName())
                .claim("price", s.getPrice())
                .compact();
        return new Token(jwt);
    }

}
```

JWTController.java

```java
@Controller
public class JWTController {

    @Autowired
    private JWTDAO jDAO;

    @RequestMapping(value = "/token.get", method = RequestMethod.GET, produces = "application/json; charset=utf-8")
    public @ResponseBody Token tokenGet(Snack s, HttpServletResponse res) {
        res.setHeader("Access-Control-Allow-Origin", "*");
        return jDAO.make(s);
    }

}
```

## 1.2.2 front-end

jwtGogogo.js

```
import axios from "axios";
import React, { useState } from "react";

const JWTGogogo = () => {
  const [name, setName] = useState("");
  const [price, setPrice] = useState("");
  const get = () => {
    axios
      .get("http://192.168.0.125/sc/token.get?name=" + name + "&price=" + price)
      .then((res) => {
        sessionStorage.setItem("token", res.data.token);
        setName("");  setPrice("");
        alert(JSON.stringify(res.data));
      });
  };
  return (
    <>
      이름
      <input value={name}
        onChange={(e) => {
          setName(e.target.value);
        }}
      />
      <br />
      가격
      <input value={price}
        onChange={(e) => {
          setPrice(e.target.value);
        }}
      />
      <hr />
      <button onClick={get}>토큰 받고</button>
    </>
  );
};

export default JWTGogogo;
```

# 1.3 JWT 확인

## 1.3.1 back-end

JWTDAO.java

```java
@Service
public class JWTDAO {

    public Token make(Snack s) {

    public Snack check(HttpServletRequest req) {
        try {
            String token = req.getParameter("token");

            Jws<Claims> jws = Jwts.parser().setSigningKey("dhffpxlqlahtQhk").parseClaimsJws(token);
            Claims c = jws.getBody();
            String name = (String) c.get("name");
            int price = (Integer) c.get("price");

            System.out.println(token);
            System.out.println(name);
            System.out.println(price);

            return new Snack(name, price);
        } catch (Exception e) {
            e.printStackTrace();
            return new Snack("없", 0);
        }
    }

}
```

JWTController.java

```java
@Controller
public class JWTController {

    @Autowired
    private JWTDAO jDAO;

    public @ResponseBody Token tokenGet(Snack s, HttpServletResponse res) {

    @RequestMapping(value = "/token.check", method = RequestMethod.GET, produces = "application/json; charset=utf-8")
    public @ResponseBody Snack tokenCheck(HttpServletRequest req, HttpServletResponse res) {
        res.setHeader("Access-Control-Allow-Origin", "*");
        return jDAO.check(req);
    }
}
```

## 1.3.2 front-end

jwtGogogo.js

```
import axios from "axios";
import React, { useState } from "react";

const JWTGogogo = () => {
  const [name, setName] = useState("");
  const [price, setPrice] = useState("");
  const get = () => {…
  };
  const check = () => {
    axios
      .get(
        "http://192.168.0.125/sc/token.check?token=" +
          sessionStorage.getItem("token")
      )
      .then((res) => {
        alert(res.data.name + " : " + res.data.price);
      });
  };
  return (
    <>
      이름 <input value={name} onChange={(e) => { setName(e.target.value); }}/>
      <br />
      가격 <input value={price} onChange={(e) => { setPrice(e.target.value); }}/>
      <hr />
      <button onClick={get}>토큰 받고</button>
      <hr/>
      <button onClick={check}>확인하고</button>
    </>
  );
};

export default JWTGogogo;
```

# 1.4 JWT 갱신

## 1.4.1 back-end

JWTDAO.java

```
@Service
public class JWTDAO {

    public Token make(Snack s) {

    public Snack check(HttpServletRequest req) {

    public Token update(HttpServletRequest req) {
        try {
            String token = req.getParameter("token");
            Jws<Claims> jws = Jwts.parser().setSigningKey("dhffpxlqlahtQhk").parseClaimsJws(token);
            Claims c = jws.getBody();

            Date now = new Date();
            String jwt = Jwts.builder()
                    .signWith(SignatureAlgorithm.HS256, "dhffpxlqlahtQhk")
                    .setExpiration(new Date(now.getTime() + Duration.ofSeconds(10).toMillis()))
                    .claim("name", c.get("name"))
                    .claim("price", c.get("price"))
                    .compact();

            return new Token(jwt);
        } catch (Exception e) {
            e.printStackTrace();
            return new Token("없어");
        }
    }

}
```

JWTController.java

```
@Controller
public class JWTController {

    @Autowired
    private JWTDAO jDAO;

    public @ResponseBody Token tokenGet(Snack s, HttpServletResponse res) {

    public @ResponseBody Snack tokenCheck(HttpServletRequest req, HttpServletResponse res) {

    @RequestMapping(value = "/token.update", method = RequestMethod.GET, produces = "application/json; charset=utf-8")
    public @ResponseBody Token tokenUpdate(HttpServletRequest req, HttpServletResponse res) {
        res.setHeader("Access-Control-Allow-Origin", "*");
        return jDAO.update(req);
    }

}
```

## 1.4.2 front-end

jwtGogogo.js

```
import axios from "axios";
import React, { useState } from "react";

const JWTGogogo = () => {
  const [name, setName] = useState("");
  const [price, setPrice] = useState("");
  const get = () => {…
  };
  const check = () => {…
  };
  const update = () => {
    axios
      .get(
        "http://192.168.0.125/sc/token.update?token=" +
          sessionStorage.getItem("token")
      )
      .then((res) => {
        if (res.data.token === "없어") {
          sessionStorage.removeItem("token");
          alert("없어");
        } else {
          sessionStorage.setItem("token", res.data.token);
        }
      });
  };
  return (
    <>
      이름 <input value={name} onChange={(e) => { setName(e.target.value); }}/><br />
      가격 <input value={price} onChange={(e) => { setPrice(e.target.value); }}/><hr />
      <button onClick={get}>토큰 받고</button><hr/>
      <button onClick={check}>확인하고</button><hr />
      <button onClick={update}>갱신하고</button>
    </>
  );
};

export default JWTGogogo;
```

## 1.5 JWT 삭제

### 1.5.1 front-end

```
import axios from "axios";
import React, { useState } from "react";

const JWTGogogo = () => {
  const [name, setName] = useState("");
  const [price, setPrice] = useState("");
  const get = () => {…
  };
  const check = () => {…
  };
  const update = () => {…
  };
  const clear = () => {
    sessionStorage.removeItem("token");
  };
  return (
    <>
      이름 <input value={name} onChange={(e) => { setName(e.target.value); }}/>
      <br />
      가격 <input value={price} onChange={(e) => { setPrice(e.target.value); }}/>
      <hr />
      <button onClick={get}>토큰 받고</button>
      <hr/>
      <button onClick={check}>확인하고</button>
      <hr />
      <button onClick={update}>갱신하고</button>
      <hr />
      <button onClick={clear}>킬</button>
    </>
  );
};

export default JWTGogogo;
```